



ANÁLISIS DEL CATÁLOGO DE SPOTIFY

Informe de proyecto — Materia: Data Experience

Autores: Jhoan Sebastián · Jofran Rodríguez

Introducción

Este informe documenta el desarrollo de un proyecto de análisis de datos sobre un catálogo de canciones de Spotify. El objetivo fue tomar un archivo CSV con datos crudos —tal como los entrega la plataforma— y, mediante un único script en Python, limpiarlos, organizarlos y convertirlos en un archivo de Excel profesional, con tablas dinámicas, fórmulas que se recalculan automáticamente y un dashboard visual con gráficos, sin necesidad de ningún paso manual intermedio.

El dataset original contiene información de 8.582 canciones, con datos como el nombre de la canción y del artista, el álbum al que pertenece, la fecha de lanzamiento, la popularidad, la duración, los géneros del artista y la cantidad de seguidores, entre otros campos. A continuación se explica el proceso completo: las decisiones de limpieza tomadas, el código desarrollado y los resultados obtenidos.

Proceso realizado

Parte 1 — Limpieza y preparación de los datos (Python / pandas)

El script inicia leyendo el archivo CSV original con pandas. A partir de ahí se aplican, en orden, los siguientes pasos de limpieza y transformación:

- Limpieza de texto: se eliminan espacios sobrantes en los campos de nombre de canción, artista, álbum, géneros y tipo de álbum.
- Valores faltantes: los artistas sin nombre (3 filas) se completan como “Desconocido”; los géneros faltantes (cerca del 39% de las filas) se completan como “Sin género registrado”, en vez de dejarse vacíos.
- Género principal: se extrae el primer género de la lista de géneros de cada artista, y se normaliza además la lista completa de géneros con un separador consistente.
- Fechas: la fecha de lanzamiento se convierte de texto a un formato de fecha real, y se extrae el año como columna independiente para facilitar los análisis por periodo.
- Traducción de categorías: el tipo de álbum (álbum / sencillo / recopilación) y el campo de contenido explícito (Sí / No) se traducen al español.

- Detección de posibles duplicados: se marca como posible duplicado cuando el mismo título de canción y el mismo artista aparecen en más de una fila.

Con los datos ya limpios se construye la tabla final (con nombres de columna en español) y se generan tres archivos: el dataset limpio completo, un top 20 de artistas por número de seguidores y un top 15 de los géneros más frecuentes. Estos dos últimos se calculan una sola vez en Python, en vez de dejarlos como fórmulas dinámicas en Excel.

Parte 2 — Construcción automática del archivo Excel (openpyxl)

La segunda parte del script toma los datos ya limpios (todavía en memoria) y construye el archivo Spotify_Analisis.xlsx completo, hoja por hoja:

- Hoja “Datos”: la tabla limpia completa, convertida en una tabla real de Excel (con encabezados con formato, filtros y ancho de columna automático).
- Hoja “Notas”: documentación de qué se limpió y qué decisiones se tomaron, para que cualquier persona que abra el archivo entienda de dónde salen los números.
- Hojas de resumen (Tabla_Anios, Tabla_TipoAlbum, Tabla_Artistas, Tabla_Generos): construidas con fórmulas de Excel (CONTAR.SI, PROMEDIO.SI, SUMAPRODUCTO, INDICE/COINCIDIR), de forma que se recalculan solas si se editan o agregan filas en la hoja de Datos.
- Hoja “Resumen”: un pequeño dashboard con tarjetas de indicadores clave (KPIs) y gráficos nativos de Excel, completamente editables, construidos sobre las hojas de resumen anteriores.

Decisiones tomadas y justificación

Rellenar en vez de eliminar. En vez de borrar las filas con artista o género faltante, se optó por rellenarlas con una categoría explícita. Eliminar el 39% de los datos por falta de género habría significado perder información valiosa de canciones y artistas reales.

No eliminar los posibles duplicados. Las filas marcadas corresponden a la misma canción publicada en distintos lanzamientos, cada una con su propio ID. Se decidió marcarlas en vez de eliminarlas, para que el usuario final

decida si excluirlas.

Top de artistas y géneros calculado en Python. Excel y LibreOffice no calculan de forma confiable listas de “top N únicos” con fórmulas dinámicas. Por eso el orden y la selección del top 20 de artistas y el top 15 de géneros se decide una sola vez en Python, mientras que las métricas de cada uno sí quedan como fórmulas en vivo.

Traducción y formato en español. Como el informe y el dashboard están dirigidos a un público hispanohablante, se tradujeron los encabezados y los valores categóricos para que el archivo final sea más claro de leer.

Código desarrollado

A continuación se presenta el script completo en Python, dividido en sus dos partes: limpieza de datos y construcción del archivo Excel.

```
"""
Pipeline de Spotify: limpieza de datos + creación del Excel final.
"""
import pandas as pd
import numpy as np
from openpyxl import Workbook
from openpyxl.worksheet.table import Table, TableStyleInfo
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side, NamedStyle
from openpyxl.utils import get_column_letter
from openpyxl.chart import BarChart, PieChart, LineChart, Reference
from openpyxl.chart.label import DataLabelList
from openpyxl.chart.marker import DataPoint
from openpyxl.comments import Comment
from openpyxl.worksheet.pagebreak import Break

RAW = "spotify_Datos_Sin_Limpiar.csv"

# =====
# PARTE 1 - Limpieza de datos
# =====
df = pd.read_csv(RAW)

# Limpiar espacios en columnas de texto
text_cols = ["track_name", "artist_name", "album_name", "artist_genres", "album_type"]
for c in text_cols:
    df[c] = df[c].astype("string").str.strip()

# Rellenar artistas sin nombre
n_missing_artist = df["artist_name"].isna().sum()
df["artist_name"] = df["artist_name"].fillna("Desconocido")

# Rellenar géneros vacíos
n_missing_genre = df["artist_genres"].isna().sum()
df["artist_genres"] = df["artist_genres"].fillna("Sin género registrado")

# Género principal = el primero de la lista
df["genero_principal"] = df["artist_genres"].apply(
    lambda s: s.split(",")[0].strip() if s != "Sin género registrado" else "Sin género"
)

# Lista completa de géneros, ya limpia
df["todos_generos"] = df["artist_genres"].apply(
    lambda s: ", ".join([g.strip() for g in s.split(",")]) if s != "Sin género registrado" else s
)

# Convertir fecha y sacar el año
df["fecha_lanzamiento"] = pd.to_datetime(df["album_release_date"], errors="coerce")
df["anio"] = df["fecha_lanzamiento"].dt.year

# Traducir tipo de álbum
tipo_map = {"album": "Álbum", "single": "Sencillo", "compilation": "Recopilación"}
df["tipo_album_es"] = df["album_type"].str.lower().map(tipo_map).fillna(df["album_type"])

# Traducir explícito a Sí/No
df["explicito_es"] = df["explicit"].map({True: "Sí", False: "No"})

# Marcar canciones que podrían estar repetidas
dup_counts = df.groupby(["track_name", "artist_name"])["track_id"].transform("count")
df["posible_duplicado"] = np.where(dup_counts > 1, "Sí", "No")

# Tabla final con nombres de columna en español
clean = pd.DataFrame({
    "Título": df["track_name"],
    "Artista": df["artist_name"],
    "Álbum": df["album_name"],
    "Tipo de Álbum": df["tipo_album_es"],
    "Fecha de Lanzamiento": df["fecha_lanzamiento"],
    "Año": df["anio"],
    "Nº Pista": df["track_number"],
    "Total Pistas Álbum": df["album_total_tracks"],
    "Duración (min)": df["track_duration_min"].round(2),
    "Popularidad Canción": df["track_popularity"],
    "Explícito": df["explicito_es"],
    "Popularidad Artista": df["artist_popularity"],
    "Seguidores Artista": df["artist_followers"],
    "Género Principal": df["genero_principal"],
    "Todos los Géneros": df["todos_generos"],
    "Posible Duplicado": df["posible_duplicado"],
    "ID Canción": df["track_id"],
```

```

})

clean = clean.sort_values(["Popularidad Canción"], ascending=False).reset_index(drop=True)

clean.to_csv("spotify_clean.csv", index=False)

# Top 20 artistas por seguidores
top_artistas = (
    df.groupby("artist_name", as_index=False)["artist_followers"]
    .max()
    .sort_values("artist_followers", ascending=False)
    .head(20)
    .reset_index(drop=True)
)
top_artistas.columns = ["Artista", "Seguidores"]
top_artistas.to_csv("top_artistas.csv", index=False)

# Top 15 géneros más frecuentes
exploded = df.assign(g=df["artist_genres"].str.split(",")).explode("g")
exploded["g"] = exploded["g"].str.strip()
exploded = exploded[exploded["g"] != "Sin género registrado"]
top_generos = exploded["g"].value_counts().head(15).reset_index()
top_generos.columns = ["Género", "Conteo"]
top_generos.to_csv("top_generos.csv", index=False)

# Resumen en consola
print("Filas:", len(df))
print("Artistas únicos:", df["artist_name"].nunique())
print("Álbumes únicos:", df["album_name"].nunique())
print("Artistas faltantes rellenados:", n_missing_artist)
print("Géneros faltantes rellenados:", n_missing_genre)
print("Posibles duplicados (título+artista repetido):",
      (df.groupby(['track_name', 'artist_name'])['track_id'].transform('count')>1).sum())
print("Rango de años:", df["anio"].min(), "-", df["anio"].max())
print("OK - archivos generados: spotify_clean.csv, top_artistas.csv, top_generos.csv")

# =====
# PARTE 2 - Construcción del Excel
# =====

# Paleta de colores
BLUE = "2A78D6"
ORANGE = "EB6834"
AQUA = "1BAF7A"
YELLOW = "EDA100"
MAGENTA = "E87BA4"
GREEN = "008300"
VIOLET = "4A3AA7"
RED = "E34948"
PALETTE = [BLUE, ORANGE, AQUA, YELLOW, MAGENTA, GREEN, VIOLET, RED]

SPOTIFY_GREEN = "1DB954" # color de marca, solo para títulos
INK = "0B0B0B"
MUTED = "52514E"
SURFACE = "FCFCFB"
HEADER_TEXT = "FFFFFF"

FONT_NAME = "Arial"

wb = Workbook()

# =====
# Estilos reutilizables
# =====
def style_title(ws, cell, text, size=16, color=SPOTIFY_GREEN, font_color="FFFFFF"):
    ws[cell] = text
    ws[cell].font = Font(name=FONT_NAME, size=size, bold=True, color=font_color)
    ws[cell].fill = PatternFill("solid", fgColor=color)

def kpi_card(ws, top_row, left_col, label, formula, number_format="#,##0", accent=BLUE):
    """Tarjeta KPI: etiqueta arriba, valor grande abajo."""
    lc = get_column_letter(left_col)
    rc = get_column_letter(left_col + 1)
    ws.merge_cells(f"{lc}{top_row}:{rc}{top_row}")
    ws.merge_cells(f"{lc}{top_row+1}:{rc}{top_row+1}")
    lab = ws[f"{lc}{top_row}"]
    lab.value = label
    lab.font = Font(name=FONT_NAME, size=10, bold=True, color="FFFFFF")
    lab.fill = PatternFill("solid", fgColor=accent)
    lab.alignment = Alignment(horizontal="center", vertical="center")
    val = ws[f"{lc}{top_row+1}"]
    val.value = formula
    val.number_format = number_format
    val.font = Font(name=FONT_NAME, size=18, bold=True, color=INK)
    val.fill = PatternFill("solid", fgColor="F2F2F0")
    val.alignment = Alignment(horizontal="center", vertical="center")

```

```

thin = Side(style="thin", color="D9D9D6")
for r in (top_row, top_row + 1):
    for c in (left_col, left_col + 1):
        ws.cell(row=r, column=c).border = Border(left=thin, right=thin, top=thin, bottom=thin)

def write_table(ws, df, start_row=1, start_col=1, table_name="Tabla", style="TableStyleMedium9",
               date_cols=None, number_formats=None, freeze=True):
    date_cols = date_cols or []
    number_formats = number_formats or {}
    ncols = len(df.columns)
    nrows = len(df)
    header_row = start_row
    for j, col in enumerate(df.columns):
        c = ws.cell(row=header_row, column=start_col + j, value=col)
        c.font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
        c.fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
        c.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
    for i, row in enumerate(df.itertuples(index=False)):
        for j, val in enumerate(row):
            cell = ws.cell(row=header_row + 1 + i, column=start_col + j)
            colname = df.columns[j]
            if colname in date_cols and pd.notna(val):
                cell.value = val.to_pydatetime() if hasattr(val, "to_pydatetime") else val
                cell.number_format = "dd/mm/yyyy"
            elif pd.isna(val):
                cell.value = None
            else:
                cell.value = val
                cell.font = Font(name=FONT_NAME, size=10, color=INK)
                if colname in number_formats:
                    cell.number_format = number_formats[colname]
    end_row = header_row + nrows
    end_col = start_col + ncols - 1
    ref = f"{get_column_letter(start_col)}{header_row}:{get_column_letter(end_col)}{end_row}"
    tab = Table(displayName=table_name, ref=ref)
    tab.tableStyleInfo = TableStyleInfo(name=style, showRowStripes=True, showFirstColumn=False)
    ws.add_table(tab)
    for j, col in enumerate(df.columns):
        maxlen = max([len(str(col))] + [len(str(v)) for v in df.iloc[:, j].astype(str).values[:200]])
        ws.column_dimensions[get_column_letter(start_col + j)].width = min(max(maxlen + 2, 10), 42)
    if freeze:
        ws.freeze_panes = ws.cell(row=header_row + 1, column=start_col)
    return ref, header_row, end_row

# =====
# HOJA "Datos" - tabla completa ya limpia
# =====
ws_datos = wb.active
ws_datos.title = "Datos"
ws_datos.sheet_view.showGridLines = False

num_fmt = {
    "N° Pista": "0",
    "Total Pistas Álbum": "0",
    "Duración (min)": "0.00",
    "Popularidad Canción": "0",
    "Popularidad Artista": "0",
    "Seguidores Artista": "#,##0",
    "Año": "0",
}

ref, hdr_row, end_row = write_table(
    ws_datos, clean, start_row=1, start_col=1, table_name="Datos",
    style="TableStyleMedium9", date_cols=["Fecha de Lanzamiento"], number_formats=num_fmt
)
N_DATOS = end_row - hdr_row # cantidad de filas de datos

# =====
# HOJA "Notas" - documentación de limpieza y supuestos
# =====
ws_notas = wb.create_sheet("Notas")
ws_notas.sheet_view.showGridLines = False
ws_notas.column_dimensions["A"].width = 100
style_title(ws_notas, "A1", "Notas de limpieza y supuestos", size=14)
ws_notas.row_dimensions[1].height = 24
notas = [
    "",
    f"Fuente original: archivo CSV subido por el usuario ({len(clean)} filas, 15 columnas originales).",
    "No se encontraron filas ni ID de canción (track_id) duplicados exactos en los datos originales.",
    "Se tradujeron encabezados y valores categóricos al español (Tipo de Álbum, Explícito).",
    "Columna 'Artista': 3 filas no tenían artista registrado; se completaron como 'Desconocido'.",
    "Columna 'Género Principal' / 'Todos los Géneros': 3.361 filas (≈39%) no tenían género registrado en el "
    "origen (Spotify no siempre asigna géneros a nivel de artista); se marcaron como 'Sin género' / "
    "'Sin género registrado' en vez de dejarse en blanco, para que los filtros y fórmulas funcionen bien.",
    "'Género Principal' = primer género de la lista de géneros del artista (columna original 'artist_genres', "
    "que puede traer varios géneros separados por coma).",
    "'Posible Duplicado' = 'Si' cuando el mismo título de canción y el mismo artista aparecen en más de una "

```

```

"fila (por ejemplo, la misma canción publicada en un sencillo y luego en un álbum o recopilación). "
"No se eliminaron estas filas porque corresponden a lanzamientos (álbum) distintos y con ID de canción "
"propio; se dejaron marcadas para que el usuario decida si quiere excluirlas de sus propios análisis.",
"'Fecha de lanzamiento' se convirtió a formato fecha (dd/mm/aaaa); 'Año' es la fecha extraída como número "
"para facilitar los filtros y las tablas por año.",
"""
"Tablas de resumen (hojas Resumen, Tabla_Años, Tabla_TipoAlbum): construidas con fórmulas "
"(COUNTIFS/AVERAGEIFS/SUMPRODUCT/INDEX-MATCH) sobre la tabla 'Datos', por lo que se recalculan solas "
"si se editan o agregan filas en 'Datos'.",
"Tabla_Artistas: el TOP 20 de artistas por seguidores se determinó una sola vez en Python (deduplicando "
"artistas), porque Excel/LibreOffice no calculan de forma fiable listas 'top N únicos' con fórmulas "
"dinámicas (funciones como UNIQUE/SORT no están disponibles de forma consistente). Las métricas de cada "
"artista (canciones, popularidad promedio, seguidores) sí son fórmulas en vivo sobre 'Datos'.",
"Tabla_Generos: el TOP 15 de géneros más frecuentes se calculó una sola vez en Python desglosando la "
"columna de géneros (una canción puede tener varios géneros a la vez). El conteo y la popularidad "
"promedio de cada género son fórmulas en vivo (SUMPRODUCT/AVERAGEIFS) sobre la columna 'Todos los "
"Géneros' de 'Datos'.",
"""
"Si agrega filas nuevas a 'Datos', esta tabla es un objeto de Excel: al escribir debajo de la última fila "
"se expande sola y las fórmulas de las hojas de resumen (que usan referencias de tabla, ej. "
"Datos[Popularidad Canción]) seguirán funcionando. El TOP de artistas y géneros (arriba explicado) no se "
"recalcula solo; para actualizarlo hay que volver a correr el script de limpieza.",
]
r = 2
for line in notas:
    cell = ws_notas.cell(row=r, column=1, value=line)
    cell.font = Font(name=FONT_NAME, size=10, color=INK)
    cell.alignment = Alignment(wrap_text=True, vertical="top")
    ws_notas.row_dimensions[r].height = 28 if line else 8
    r += 1

# =====
# HOJA "Tabla_Años" – canciones y popularidad promedio por año
# =====
ws_anios = wb.create_sheet("Tabla_Años")
ws_anios.sheet_view.showGridLines = False
style_title(ws_anios, "A1", "Canciones y popularidad promedio por año", size=13)
ws_anios.merge_cells("A1:D1")

years = sorted([y for y in clean["Año"].unique() if y >= 2000])
labels = ["Antes de 2000"] + [str(y) for y in years]
year_keys = [None] + years # None -> bucket especial "<2000"

headers = ["Periodo", "Año (num)", "Canciones", "Popularidad Promedio"]
hr = 3
for j, h in enumerate(headers, start=1):
    c = ws_anios.cell(row=hr, column=j, value=h)
    c.font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
    c.fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
    c.alignment = Alignment(horizontal="center", wrap_text=True)

for i, (lab, yk) in enumerate(zip(labels, year_keys)):
    r = hr + 1 + i
    ws_anios.cell(row=r, column=1, value=lab).font = Font(name=FONT_NAME, size=10)
    ws_anios.cell(row=r, column=2, value=yk).font = Font(name=FONT_NAME, size=10)
    if yk is None:
        f_count = '=COUNTIFS(Datos[Año], "<2000")'
        f_avg = '=IFERROR(AVERAGEIFS(Datos[Popularidad Canción], Datos[Año], "<2000"), 0)'
    else:
        f_count = f'=COUNTIFS(Datos[Año], $B{r})'
        f_avg = f'=IFERROR(AVERAGEIFS(Datos[Popularidad Canción], Datos[Año], $B{r}), 0)'
    cc = ws_anios.cell(row=r, column=3, value=f_count)
    cc.font = Font(name=FONT_NAME, size=10)
    cc.number_format = "#,##0"
    ca = ws_anios.cell(row=r, column=4, value=f_avg)
    ca.font = Font(name=FONT_NAME, size=10)
    ca.number_format = "0.0"

ws_anios.column_dimensions["A"].width = 16
ws_anios.column_dimensions["B"].width = 12
ws_anios.column_dimensions["C"].width = 14
ws_anios.column_dimensions["D"].width = 20
ws_anios.column_dimensions["B"].hidden = True
last_year_row = hr + len(labels)
ws_anios.freeze_panes = "A4"

# =====
# HOJA "Tabla_TipoAlbum" – desglose por tipo de álbum
# =====
ws_tipo = wb.create_sheet("Tabla_TipoAlbum")
ws_tipo.sheet_view.showGridLines = False
style_title(ws_tipo, "A1", "Desglose por tipo de álbum", size=13)
ws_tipo.merge_cells("A1:D1")

tipos = ["Álbum", "Sencillo", "Recopilación"]
headers = ["Tipo de Álbum", "Canciones", "% del Total", "Popularidad Promedio"]

```

```

hr = 3
for j, h in enumerate(headers, start=1):
    c = ws_tipo.cell(row=hr, column=j, value=h)
    c.font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
    c.fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
    c.alignment = Alignment(horizontal="center", wrap_text=True)

for i, t in enumerate(tipos):
    r = hr + 1 + i
    ws_tipo.cell(row=r, column=1, value=t).font = Font(name=FONT_NAME, size=10)
    cc = ws_tipo.cell(row=r, column=2, value=f'=COUNTIF(Datos[Tipo de Álbum],$A{r})')
    cc.font = Font(name=FONT_NAME, size=10)
    cc.number_format = "#,##0"
    cp = ws_tipo.cell(row=r, column=3, value=f'=B{r}/COUNTA(Datos[Título])')
    cp.font = Font(name=FONT_NAME, size=10)
    cp.number_format = "0.0%"
    ca = ws_tipo.cell(row=r, column=4, value=f'=IFERROR(AVERAGEIFS(Datos[Popularidad Canción],Datos[Tipo de
        Álbum],$A{r}),0)')
    ca.font = Font(name=FONT_NAME, size=10)
    ca.number_format = "0.0"

ws_tipo.column_dimensions["A"].width = 18
ws_tipo.column_dimensions["B"].width = 14
ws_tipo.column_dimensions["C"].width = 14
ws_tipo.column_dimensions["D"].width = 20
last_tipo_row = hr + len(tipos)

# =====
# HOJA "Tabla_Artistas" – Top 20 artistas por seguidores
# =====
ws_art = wb.create_sheet("Tabla_Artistas")
ws_art.sheet_view.showGridLines = False
style_title(ws_art, "A1", "Top 20 artistas por seguidores (fórmulas en vivo sobre 'Datos')", size=13)
ws_art.merge_cells("A1:E1")

headers = ["#", "Artista", "Seguidores", "Canciones en el dataset", "Popularidad Promedio Canción"]
hr = 3
for j, h in enumerate(headers, start=1):
    c = ws_art.cell(row=hr, column=j, value=h)
    c.font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
    c.fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
    c.alignment = Alignment(horizontal="center", wrap_text=True)

for i, row in enumerate(top_artistas.itertuples(index=False)):
    r = hr + 1 + i
    ws_art.cell(row=r, column=1, value=i + 1).font = Font(name=FONT_NAME, size=10)
    ws_art.cell(row=r, column=2, value=row[0]).font = Font(name=FONT_NAME, size=10)
    cf = ws_art.cell(row=r, column=3, value=f'=_xlfn.MAXIFS(Datos[Seguidores Artista],Datos[Artista],$B{r})')
    cf.font = Font(name=FONT_NAME, size=10)
    cf.number_format = "#,##0"
    cc = ws_art.cell(row=r, column=4, value=f'=COUNTIF(Datos[Artista],$B{r})')
    cc.font = Font(name=FONT_NAME, size=10)
    cc.number_format = "#,##0"
    ca = ws_art.cell(row=r, column=5, value=f'=IFERROR(AVERAGEIFS(Datos[Popularidad Canción],Datos[Artista],$B{r}),0)')
    ca.font = Font(name=FONT_NAME, size=10)
    ca.number_format = "0.0"

ws_art.column_dimensions["A"].width = 6
ws_art.column_dimensions["B"].width = 26
ws_art.column_dimensions["C"].width = 14
ws_art.column_dimensions["D"].width = 20
ws_art.column_dimensions["E"].width = 24
last_art_row = hr + len(top_artistas)
ws_art.freeze_panes = "A4"

# =====
# HOJA "Tabla_Generos" – Top 15 géneros más frecuentes
# =====
ws_gen = wb.create_sheet("Tabla_Generos")
ws_gen.sheet_view.showGridLines = False
style_title(ws_gen, "A1", "Top 15 géneros más frecuentes (fórmulas en vivo sobre 'Datos')", size=13)
ws_gen.merge_cells("A1:D1")

headers = ["#", "Género", "Canciones", "Popularidad Promedio"]
hr = 3
for j, h in enumerate(headers, start=1):
    c = ws_gen.cell(row=hr, column=j, value=h)
    c.font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
    c.fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
    c.alignment = Alignment(horizontal="center", wrap_text=True)

for i, row in enumerate(top_generos.itertuples(index=False)):
    r = hr + 1 + i
    genero = row[0]
    ws_gen.cell(row=r, column=1, value=i + 1).font = Font(name=FONT_NAME, size=10)
    ws_gen.cell(row=r, column=2, value=genero).font = Font(name=FONT_NAME, size=10)

```

```

# Busca el género dentro de la lista de géneros de cada canción
search_expr = (
    f'SEARCH("{&$B{r}&"", "", "&SUBSTITUTE(Datos[Todos los Géneros], "", "&")&")'
)
f_count = f"=SUMPRODUCT(--ISNUMBER({search_expr}))"
cc = ws_gen.cell(row=r, column=3, value=f_count)
cc.font = Font(name=FONT_NAME, size=10)
cc.number_format = "#,##0"
f_avg = (
    f"=IFERROR(SUMPRODUCT(--ISNUMBER({search_expr})*Datos[Popularidad Canción])/C{r},0)"
)
ca = ws_gen.cell(row=r, column=4, value=f_avg)
ca.font = Font(name=FONT_NAME, size=10)
ca.number_format = "0.0"

ws_gen.column_dimensions["A"].width = 6
ws_gen.column_dimensions["B"].width = 22
ws_gen.column_dimensions["C"].width = 14
ws_gen.column_dimensions["D"].width = 20
last_gen_row = hr + len(top_generos)
ws_gen.freeze_panes = "A4"

# =====
# HOJA "Resumen" – Dashboard con KPIs y gráficos
# =====
ws_res = wb.create_sheet("Resumen", 0) # primera hoja
ws_res.sheet_view.showGridLines = False
ws_res.sheet_properties.tabColor = SPOTIFY_GREEN

style_title(ws_res, "B2", " Análisis del catálogo de Spotify", size=20)
ws_res.merge_cells("B2:K2")
ws_res.row_dimensions[2].height = 32
sub = ws_res["B3"]
sub.value = f"{{len(clean:)} canciones · {{clean['Artista'].nunique():}} artistas · {{clean['Álbum'].nunique():}} álbumes · datos limpiados y organizados automáticamente"
sub.font = Font(name=FONT_NAME, size=10, italic=True, color=MUTED)
ws_res.merge_cells("B3:K3")
# ---- KPI cards (fila 5-6) ----
kpi_row = 5
kpi_card(ws_res, kpi_row, 2, "TOTAL CANCIONES", "=COUNTA(Datos[Título])", "#,##0", accent=BLUE)
# Escapa caracteres especiales para que COUNTIF compare bien (ej: "NSYNC")
esc = lambda col: f'="{&SUBSTITUTE(SUBSTITUTE(SUBSTITUTE(Datos[{col}], "~", "~"), "*", "~*"), "?", "~?")}'
kpi_card(ws_res, kpi_row, 4, "ARTISTAS ÚNICOS", f"=SUMPRODUCT(1/COUNTIF(Datos[Artista],{esc('Artista')}))", "#,##0", accent=ORANGE)
kpi_card(ws_res, kpi_row, 6, "ÁLBUMES ÚNICOS", f"=SUMPRODUCT(1/COUNTIF(Datos[Álbum],{esc('Álbum')}))", "#,##0", accent=AQUA)
kpi_card(ws_res, kpi_row, 8, "POPULARIDAD PROM. CANCIÓN", "=AVERAGE(Datos[Popularidad Canción])", "0.0", accent=VIOLET)
kpi_card(ws_res, kpi_row, 10, "DURACIÓN PROMEDIO (min)", "=AVERAGE(Datos[Duración (min)])", "0.00", accent=MAGENTA)

# Segunda fila de KPIs
kpi_row2 = 8
kpi_card(ws_res, kpi_row2, 2, "% CANCIONES EXPLÍCITAS", '=COUNTIF(Datos[Explícito], "Sí")/COUNTA(Datos[Título])', "0.0%", accent=RED)
kpi_card(ws_res, kpi_row2, 4, "RANGO DE AÑOS", '=TEXT(MIN(Datos[Fecha de Lanzamiento]), "yyyy")&" - &TEXT(MAX(Datos[Fecha de Lanzamiento]), "yyyy")', "@", accent=GREEN)
kpi_card(ws_res, kpi_row2, 6, "CANCIÓN MÁS POPULAR", "=INDEX(Datos[Título], MATCH(MAX(Datos[Popularidad Canción]), Datos[Popularidad Canción], 0))", "@", accent=BLUE)
kpi_card(ws_res, kpi_row2, 8, "ARTISTA CON MÁS SEGUIDORES", "=INDEX(Datos[Artista], MATCH(MAX(Datos[Seguidores Artista]), Datos[Seguidores Artista], 0))", "@", accent=ORANGE)
kpi_card(ws_res, kpi_row2, 10, "POSIBLES DUPLICADOS", '=COUNTIF(Datos[Posible Duplicado], "Sí")', "#,##0", accent=AQUA)

for col in range(2, 12):
    ws_res.column_dimensions[get_column_letter(col)].width = 13.5

# =====
# Gráficos nativos (en Resumen), usando la paleta validada
# =====
def style_series(series, color):
    series.graphicalProperties.solidFill = color
    series.graphicalProperties.line.noFill = True

chart_anchor_row = 12

# 1) Canciones por año (barras)
bar_year = BarChart()
bar_year.type = "col"
bar_year.title = "Canciones publicadas por año"
bar_year.y_axis.title = "Canciones"
bar_year.x_axis.title = None
bar_year.style = None
bar_year.height = 8.5
bar_year.width = 17
data = Reference(ws_anios, min_col=3, min_row=3, max_row=last_year_row)
cats = Reference(ws_anios, min_col=1, min_row=4, max_row=last_year_row)
bar_year.add_data(data, titles_from_data=True)
bar_year.set_categories(cats)

```

```

style_series(bar_year.series[0], BLUE)
bar_year.legend = None
bar_year.gapWidth = 30
ws_res.add_chart(bar_year, f"B{chart_anchor_row}")

# 2) Popularidad promedio por año (líneas)
line_pop = LineChart()
line_pop.title = "Popularidad promedio por año"
line_pop.y_axis.title = "Popularidad (0-100)"
line_pop.height = 8.5
line_pop.width = 17
data = Reference(ws_anios, min_col=4, min_row=3, max_row=last_year_row)
cats = Reference(ws_anios, min_col=1, min_row=4, max_row=last_year_row)
line_pop.add_data(data, titles_from_data=True)
line_pop.set_categories(cats)
line_pop.series[0].graphicalProperties.line.solidFill = ORANGE
line_pop.series[0].graphicalProperties.line.width = 20000
line_pop.series[0].smooth = False
line_pop.legend = None
ws_res.add_chart(line_pop, f"G{chart_anchor_row}")

chart_row2 = chart_anchor_row + 18

# 3) Distribución por tipo de álbum (pastel)
pie_tipo = PieChart()
pie_tipo.title = "Distribución por tipo de álbum"
pie_tipo.height = 8.5
pie_tipo.width = 11
data = Reference(ws_tipo, min_col=2, min_row=3, max_row=last_tipo_row)
cats = Reference(ws_tipo, min_col=1, min_row=4, max_row=last_tipo_row)
pie_tipo.add_data(data, titles_from_data=True)
pie_tipo.set_categories(cats)
pts = []
for i, color in enumerate([BLUE, ORANGE, AQUA]):
    dp = DataPoint(idx=i)
    dp.graphicalProperties.solidFill = color
    pts.append(dp)
pie_tipo.series[0].data_points = pts
pie_tipo.dataLabels = DataLabelList()
pie_tipo.dataLabels.showPercent = True
pie_tipo.dataLabels.showCatName = False
pie_tipo.dataLabels.showSerName = False
pie_tipo.dataLabels.showVal = False
pie_tipo.dataLabels.showLegendKey = False
pie_tipo.legend.position = "b"
ws_res.add_chart(pie_tipo, f"B{chart_row2}")

# 4) Contenido explícito vs no explícito (pastel, con tabla auxiliar)
ws_tipo["F3"] = "Explicito"
ws_tipo["F3"].font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
ws_tipo["F3"].fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
ws_tipo["G3"] = "Canciones"
ws_tipo["G3"].font = Font(name=FONT_NAME, size=10, bold=True, color=HEADER_TEXT)
ws_tipo["G3"].fill = PatternFill("solid", fgColor=SPOTIFY_GREEN)
ws_tipo["F4"] = "Sí"
ws_tipo["F5"] = "No"
ws_tipo["G4"] = '=COUNTIF(Datos[Explicito],$F4)'
ws_tipo["G5"] = '=COUNTIF(Datos[Explicito],$F5)'
for rr in (4, 5):
    ws_tipo.cell(row=rr, column=6).font = Font(name=FONT_NAME, size=10)
    ws_tipo.cell(row=rr, column=7).font = Font(name=FONT_NAME, size=10)
    ws_tipo.cell(row=rr, column=7).number_format = "#,##0"
ws_tipo.column_dimensions["F"].width = 12
ws_tipo.column_dimensions["G"].width = 12

pie_exp = PieChart()
pie_exp.title = "Contenido explícito vs. no explícito"
pie_exp.height = 8.5
pie_exp.width = 11
data = Reference(ws_tipo, min_col=7, min_row=3, max_row=5)
cats = Reference(ws_tipo, min_col=6, min_row=4, max_row=5)
pie_exp.add_data(data, titles_from_data=True)
pie_exp.set_categories(cats)
pts2 = []
for i, color in enumerate([RED, AQUA]):
    dp = DataPoint(idx=i)
    dp.graphicalProperties.solidFill = color
    pts2.append(dp)
pie_exp.series[0].data_points = pts2
pie_exp.dataLabels = DataLabelList()
pie_exp.dataLabels.showPercent = True
pie_exp.dataLabels.showCatName = False
pie_exp.dataLabels.showSerName = False
pie_exp.dataLabels.showVal = False
pie_exp.dataLabels.showLegendKey = False
pie_exp.legend.position = "b"

```

```

ws_res.add_chart(pie_exp, f"E{chart_row2}")

# 5) Top 15 géneros (barras horizontales)
bar_gen = BarChart()
bar_gen.type = "bar"
bar_gen.title = "Top 15 géneros más frecuentes"
bar_gen.height = 8.5
bar_gen.width = 17
data = Reference(ws_gen, min_col=3, min_row=3, max_row=last_gen_row)
cats = Reference(ws_gen, min_col=2, min_row=4, max_row=last_gen_row)
bar_gen.add_data(data, titles_from_data=True)
bar_gen.set_categories(cats)
style_series(bar_gen.series[0], VIOLET)
bar_gen.legend = None
bar_gen.gapWidth = 30
bar_gen.y_axis.delete = False
ws_res.add_chart(bar_gen, f"H{chart_row2}")

chart_row3 = chart_row2 + 18

# 6) Top 10 artistas por seguidores (barras horizontales)
bar_art = BarChart()
bar_art.type = "bar"
bar_art.title = "Top 10 artistas por seguidores"
bar_art.height = 9
bar_art.width = 17
data = Reference(ws_art, min_col=3, min_row=3, max_row=13)
cats = Reference(ws_art, min_col=2, min_row=4, max_row=13)
bar_art.add_data(data, titles_from_data=True)
bar_art.set_categories(cats)
style_series(bar_art.series[0], GREEN)
bar_art.legend = None
bar_art.gapWidth = 30
bar_art.y_axis.delete = False
ws_res.add_chart(bar_art, f"B{chart_row3}")

wb.save("Spotify_Analisis.xlsx")
print("Workbook completo guardado. last_year_row=", last_year_row, "last_tipo_row=", last_tipo_row,
      "last_art_row=", last_art_row, "last_gen_row=", last_gen_row)

```

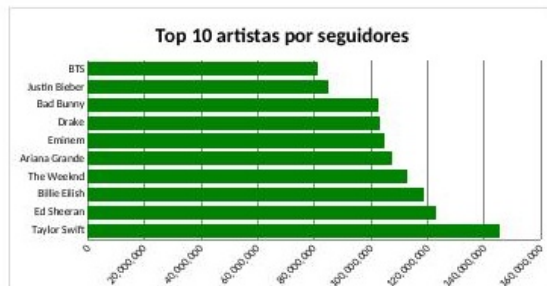
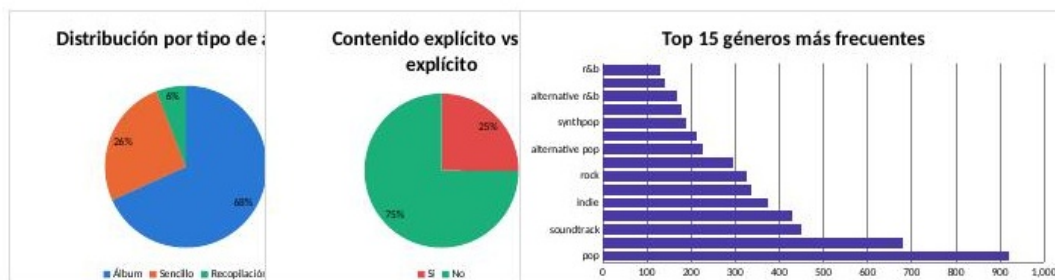
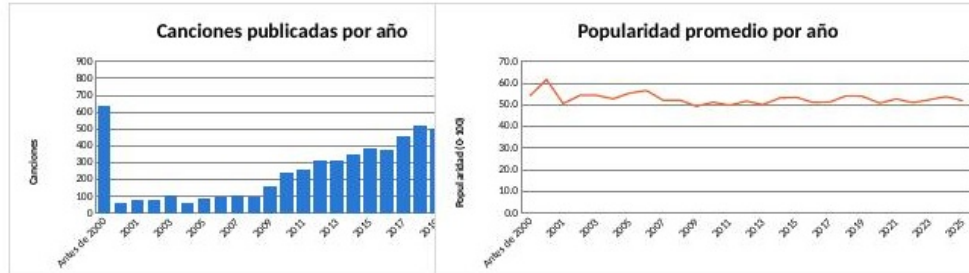
Resultados obtenidos

El archivo Spotify_Analisis.xlsx generado incluye un dashboard (hoja “Resumen”) con los indicadores clave del catálogo y seis gráficos nativos de Excel, contruidos sobre los datos ya limpios:

Análisis del catálogo de Spotify

8,582 canciones · 2,548 artistas · 4,870 álbumes · datos limpiados y organizados automáticamente

TOTAL CANCIONES	ARTISTAS ÚNICOS	ÁLBUMES ÚNICOS	POPULARIDAD PROM. CANCIÓN	DURACIÓN PROMEDIO (min)
8,582	2,548	4,829	52.4	3.49
% CANCIONES EXPLÍCITAS	RANGO DE AÑOS	CANCIÓN MÁS POPULAR	ARTISTA CON MÁS SEGUIDORES	POSIBLES DUPLICADOS
25.0%	1952 - 2025	Golden	Taylor Swift	1,237



Dashboard generado automáticamente en la hoja "Resumen" del archivo Excel.

Lectura de los indicadores clave

- El dataset limpio quedó con 8.582 canciones, de 2.548 artistas únicos y 4.829 álbumes únicos.
- La popularidad promedio de las canciones es de 52.4 puntos (sobre 100), y la duración promedio es de 3.49 minutos por canción.
- El 25% de las canciones tiene contenido explícito, mientras que el 75% no lo tiene.
- El catálogo cubre un rango de años muy amplio, de 1952 a 2025.
- La canción más popular del dataset es “Golden”, y el artista con más seguidores es Taylor Swift.
- Se detectaron 1.237 posibles duplicados (misma canción y artista en más de un lanzamiento).

Lectura de los gráficos

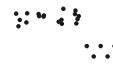
- Canciones publicadas por año: el catálogo muestra un crecimiento sostenido de lanzamientos a partir de los años 2010, con un salto notorio en los últimos años del dataset.
- Popularidad promedio por año: se mantiene relativamente estable desde 2016 en adelante, entre 50 y 54 puntos, sin caídas ni subidas bruscas.
- Distribución por tipo de álbum: la mayoría de las canciones (68%) pertenece a álbumes completos, un 26% son sencillos y un 6% son recopilaciones.
- Contenido explícito vs. no explícito: confirma el 75% / 25% señalado en los indicadores clave.
- Top 15 géneros más frecuentes: el pop es, por amplio margen, el género más frecuente en el dataset, seguido de soundtrack, indie, rock, alternative pop y synthpop, entre otros.
- Top 10 artistas por seguidores: encabezado por Taylor Swift, seguida de Ed Sheeran, Billie Eilish, The Weeknd, Ariana Grande, Eminem, Drake, Bad Bunny, Justin Bieber y BTS.

Conclusiones

El proyecto permitió pasar de un archivo CSV crudo y desordenado a un entregable profesional en Excel, con datos limpios, decisiones de limpieza documentadas dentro del propio archivo, y un dashboard con indicadores y gráficos listos para presentar o seguir explorando.

El uso de fórmulas dinámicas, en vez de valores fijos, en la mayoría de las hojas de resumen garantiza que el archivo siga siendo útil si se agregan o editan canciones en el futuro, sin depender de volver a correr el script para cada actualización menor.

Este flujo de trabajo —limpieza y automatización con Python, entrega final en un formato accesible como Excel— es directamente aplicable en un contexto profesional de Ingeniería de Sistemas, especialmente en proyectos donde se necesite entregar reportes recurrentes a personas que no programan.



ean
universidad